MPOO 2027-1 · GRUPO 6 · PRÁCTICA 8 · MIÉRCOLES 21 DE OCTUBRE

La estación de clasificación

Herencia, segunda parte. Una banda transportadora con tres sensores y tres actuadores: primero la dibujas en un diagrama de clases y luego la programas en cuatro paquetes.

Descargar los archivos (.zip)

Qué vas a hacer

En pareja. Cuatro pasos:

1. **Dibuja el diagrama de clases**
   15 min · en papel o draw.io

2. **Escribe los cimientos: 4 clases**
   25 min · terminas cuando `PruebaTemperatura` imprime lo de la guía

3. **Escribe la estación: 7 clases más**
   45 min · terminas cuando `Main` imprime lo de la guía

4. **Entrega**
   viernes 23 de octubre, 23:59, en `p08/`

La historia

En una paquetería, los paquetes pasan por una **banda transportadora**. Tres **sensores** los vigilan: uno ve si hay un paquete atorado, otro lo pesa y otro mide la temperatura del motor. Tres **actuadores** responden: la banda, un **semáforo** y una **alarma**.

Si **cualquier** sensor da alerta, la banda se detiene, el semáforo se pone en rojo y suena la alarma. Si no, la banda avanza y el semáforo queda en verde.

Cada dispositivo trae **su propia batería**, que se gasta con el uso. La **estación** tiene hasta cinco sensores y cinco actuadores, y un sensor se puede sacar a mantenimiento y volver a instalar.

El mapa

Así queda todo cuando terminas. Tenlo a la vista en los cuatro pasos.

Quién hereda de quién, y en qué paquete vive

```
Dispositivo (abstract)        dispositivos  ◆ 1 Bateria
├── Sensor (abstract)         dispositivos
│   ├── SensorPresencia       sensores
│   ├── SensorPeso            sensores
│   └── SensorTemperatura     sensores
└── Actuador (abstract)       dispositivos
    ├── BandaTransportadora   actuadores
    ├── Semaforo              actuadores    ◆ 3 Luz
    └── Alarma                actuadores

Estacion   estacion      ◇ 1..5 Sensor y ◇ 1..5 Actuador
Bateria    dispositivos  ◆ la crea Dispositivo con new
Luz        dispositivos  ◆ las crea Semaforo con new
```

`├──` es **«es un»** (`extends`). ◆ **composición**: el dueño hace el `new` y la parte muere con él. ◇ **agregación**: la parte llega de afuera y sobrevive aunque la saquen.

Las carpetas en IntelliJ

```
src
├── dispositivos
│   ├── Bateria.java          paso 2
│   ├── Dispositivo.java      paso 2
│   ├── Sensor.java           paso 2
│   ├── Actuador.java         paso 3
│   └── Luz.java              viene en el .zip
├── sensores
│   ├── SensorTemperatura.javapaso 2
│   ├── SensorPresencia.java  paso 3
│   └── SensorPeso.java       paso 3
├── actuadores
│   ├── BandaTransportadora.javapaso 3
│   ├── Semaforo.java         paso 3
│   └── Alarma.java           paso 3
└── estacion
    ├── Estacion.java         paso 3
    ├── PruebaTemperatura.javaviene en el .zip
    └── Main.java             viene en el .zip, va al final
```

Para crear cada carpeta: clic derecho en `src` → **New → Package**. Cada archivo empieza con el `package` de su carpeta.

Paso 1 · El diagrama de clases

- **Una caja por clase del mapa** (son 12, sin `Main` ni `PruebaTemperatura`), con sus atributos y métodos copiados de las fichas de los pasos 2 y 3.
- **Los símbolos:** `+` public, `–` private, `#` protected, `~` sin modificador. Las clases y métodos `abstract`, en cursiva.
- **Las flechas, como en el mapa:** triángulo vacío hacia el padre; ◆ y ◇ del lado del dueño, con su número (`1`, `3`, `1..5`).

A los 15 minutos se proyecta la solución y corrigen el suyo.

Paso 2 · Los cimientos

Crea los cuatro paquetes y copia del .zip `Luz.java` (a `dispositivos`) y `PruebaTemperatura.java` (a `estacion`). **`Main.java` todavía no**: usa clases que aún no existen y nada correría.

Cómo leer una ficha: es el esqueleto de la clase. Escribe cada línea tal cual y, donde hay un método, ponle su cuerpo `{ ... }` haciendo lo que dice el comentario. Las líneas con `;` al final (los `abstract`) no llevan cuerpo.

1 · `Bateria` en dispositivos

La batería de cada dispositivo.

```
package dispositivos;

class Bateria {                          // sin public: solo la ve su paquete
    private int carga;                   // de 0 a 100

    Bateria(int carga)                   // menor que 0 → 0; mayor que 100 → 100
    int getCarga()
    void descargar(int puntos)           // resta, sin bajar de 0 (Math.max)
}
```

2 · `Dispositivo` en dispositivos

El padre de todos. Fíjate en el `new Bateria` dentro del constructor y en el segundo constructor, que solo llama al primero con `this`.

```
package dispositivos;

public abstract class Dispositivo {
    private final String id;
    private final Bateria bateria;       // ◆ composición

    public Dispositivo(String id, int carga) // guarda el id y hace new Bateria(carga)
    public Dispositivo(String id)        // this(id, 100)
    public String getId()
    public int getCarga()                // la carga de su batería
    protected void gastar(int puntos)    // descarga su batería
    @Override public String toString()   // "S3 (100%)"
}
```

3 · `Sensor` en dispositivos

Abstracta y hija de otra abstracta. Sabe **leer**, pero cada sensor decide cómo **medir**.

```
package dispositivos;

public abstract class Sensor extends Dispositivo {
    private final String unidad;
    private double senal;

    public Sensor(String id, String unidad, int carga)  // super(id, carga)
    public Sensor(String id, String unidad)             // this(id, unidad, 100)
    public String getUnidad()
    public void recibir(double senal)        // Main simula la señal
    protected double getSenal()
    public abstract double medir();
    public abstract boolean hayAlerta();
    public final String leer()               // gastar(1) y "S3 mide 40.0 C"
                                             // más " ALERTA" si hayAlerta()
    @Override public String toString()       // "Sensor " + lo del padre
}
```

4 · `SensorTemperatura` en sensores

El primer sensor de verdad. Vive en otro paquete: por eso el `import`.

```
package sensores;

import dispositivos.Sensor;

public class SensorTemperatura extends Sensor {
    private double calibracion;
    private double maxima;

    public SensorTemperatura(String id, double calibracion, double maxima)
                                             // unidad "C"
    @Override public double medir()          // señal + calibración
    @Override public boolean hayAlerta()     // medir() > maxima
    @Override public String toString()       // lo del padre + " temperatura, max 45.0"
}
```

Corre `PruebaTemperatura`:

```
package estacion;

import sensores.SensorTemperatura;

public class PruebaTemperatura {
    public static void main(String[] args) {
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        System.out.println(termo);
        termo.recibir(41.5);
        System.out.println(termo.leer());
        termo.recibir(50);
        System.out.println(termo.leer());
        System.out.println(termo);
    }
}
```

> **DEBE SALIR**
>
> ```
> Sensor S3 (100%) temperatura, max 45.0
> S3 mide 40.0 C
> S3 mide 48.5 C ALERTA
> Sensor S3 (98%) temperatura, max 45.0
> ```

El `98%` de la última línea es la batería: cada `leer()` gastó 1.

Paso 3 · La estación

Las demás clases, en este orden.

5 · `SensorPresencia` en sensores

Igual que la de temperatura, con otra forma de medir.

```
package sensores;

import dispositivos.Sensor;

public class SensorPresencia extends Sensor {

    public SensorPresencia(String id)        // unidad "cm"
    @Override public double medir()          // llega en milímetros: señal / 10
    @Override public boolean hayAlerta()     // medir() < 2: el paquete se atoró
    @Override public String toString()       // lo del padre + " presencia"
}
```

6 · `SensorPeso` en sensores

Tiene dos constructores, como `Dispositivo`.

```
package sensores;

import dispositivos.Sensor;

public class SensorPeso extends Sensor {
    private double limite;

    public SensorPeso(String id, int carga, double limite)
                                         // unidad "kg"; límite 0 o menos → 1
    public SensorPeso(String id, double limite)   // this(id, 100, limite)
    @Override public double medir()          // llega en gramos: señal / 1000
    @Override public boolean hayAlerta()     // medir() > limite
    @Override public String toString()       // lo del padre + " peso, limite 25.0"
}
```

7 · `Actuador` en dispositivos

El otro hijo de `Dispositivo`, hermano de `Sensor`.

```
package dispositivos;

public abstract class Actuador extends Dispositivo {
    private boolean encendido;              // nace en false

    public Actuador(String id, int carga)    // super(id, carga)
    public Actuador(String id)               // this(id, 100)
    public boolean isEncendido()
    public void encender()
    public void apagar()
    public abstract String reaccionar(boolean alerta);
    @Override public String toString()       // "Actuador " + lo del padre
}
```

8 · `Alarma` en actuadores

El actuador más sencillo.

```
package actuadores;

import dispositivos.Actuador;

public class Alarma extends Actuador {
    private int volumen;                     // de 0 a 10

    public Alarma(String id, int volumen)    // menor que 0 → 0; mayor que 10 → 10
    public int getVolumen()
    @Override public String reaccionar(boolean alerta)
        // con alerta: encender() y "A3 suena a volumen 10"
        // sin alerta: apagar() y "A3 en silencio"
    @Override public String toString()       // lo del padre + " alarma, volumen 10"
}
```

9 · `BandaTransportadora` en actuadores

Dos métodos `avanzar`: el que no recibe nada llama al otro.

```
package actuadores;

import dispositivos.Actuador;

public class BandaTransportadora extends Actuador {
    private int velocidad;                  // metros por ciclo; menor que 1 → 1
    private int recorrido;                  // metros acumulados

    public BandaTransportadora(String id, int velocidad)
    public int getRecorrido()
    public String avanzar()                 // return avanzar(velocidad);
    public String avanzar(int metros)
        // apagada:   "A1 apagada, no avanza"
        // encendida: suma los metros, gastar(metros) y "A1 avanza 2 m, lleva 2 m"
    @Override public String reaccionar(boolean alerta)
        // con alerta: apagar() y "A1 se detiene"
        // sin alerta: encender() y return avanzar();
    @Override public String toString()      // lo del padre + " banda, 2 m por ciclo"
}
```

10 · `Semaforo` en actuadores

Crea sus tres luces. `Luz` ya viene en el .zip.

```
package actuadores;

import dispositivos.Actuador;
import dispositivos.Luz;

public class Semaforo extends Actuador {
    private final Luz[] luces;              // ◆ composición

    public Semaforo(String id)              // 3 new Luz: "rojo", "amarillo", "verde"
    public void ponerEn(String color)       // enciende esa luz y apaga las otras
    public String getColor()                // el color de la encendida, o "apagado"
    @Override public String reaccionar(boolean alerta)
        // encender(); ponerEn("rojo") con alerta, ponerEn("verde") sin ella
        // y "A2 en verde"
    @Override public String toString()      // lo del padre + " semaforo en verde"
}
```

11 · `Estacion` en estacion

Recibe los dispositivos que crea `Main` y en cada ciclo los pone a trabajar.

```
package estacion;

import dispositivos.Actuador;
import dispositivos.Sensor;

public class Estacion {
    private final String nombre;
    private Sensor[] sensores;              // ◇ agregación: new Sensor[5], vacío
    private Actuador[] actuadores;          // ◇ agregación: new Actuador[5], vacío
    private int numSensores;
    private int numActuadores;
    private int ciclos;

    public Estacion(String nombre)
    public void instalar(Sensor s)          // en el siguiente lugar libre
    public void instalar(Actuador a)        //   y "Instalado: " + el dispositivo
                                            //   si no cabe: "Sin lugar para S1"
    public Sensor retirar(String id)
        // lo busca por id, recorre los que siguen un lugar a la izquierda,
        // "Retirado: S2" y lo devuelve; si no está, devuelve null
    public void ciclo()
        // 1. ciclos++ y "== Ciclo 1 en Anden 3"
        // 2. cada sensor: "  " + leer(); si alguno hayAlerta(), hay alerta
        // 3. cada actuador: "  " + reaccionar(alerta)
    @Override public String toString()      // "Anden 3: 3 sensores, 3 actuadores, 3 ciclos"
}
```

Con todo listo, copia `Main.java` a `estacion` y córrelo:

```
package estacion;

import actuadores.Alarma;
import actuadores.BandaTransportadora;
import actuadores.Semaforo;
import dispositivos.Sensor;
import sensores.SensorPeso;
import sensores.SensorPresencia;
import sensores.SensorTemperatura;

public class Main {
    public static void main(String[] args) {
        // los dispositivos nacen aquí, fuera de la estación
        SensorPresencia ojo = new SensorPresencia("S1");
        SensorPeso bascula = new SensorPeso("S2", 80, 25);
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        BandaTransportadora banda = new BandaTransportadora("A1", 2);
        Semaforo semaforo = new Semaforo("A2");
        Alarma alarma = new Alarma("A3", 15);

        Estacion anden = new Estacion("Anden 3");
        anden.instalar(ojo);
        anden.instalar(bascula);
        anden.instalar(termo);
        anden.instalar(banda);
        anden.instalar(semaforo);
        anden.instalar(alarma);

        // un paquete normal
        ojo.recibir(120);
        bascula.recibir(18500);
        termo.recibir(41.5);
        anden.ciclo();

        // un paquete con sobrepeso
```

```
        ojo.recibir(95);
        bascula.recibir(32000);
        termo.recibir(43.5);
        anden.ciclo();

        // la báscula sale a mantenimiento y el motor se calienta
        Sensor fuera = anden.retirar("S2");
        ojo.recibir(150);
        termo.recibir(48);
        anden.ciclo();

        // la báscula regresa: es el mismo objeto
        System.out.println("Regresa: " + fuera);
        anden.instalar(fuera);
        banda.encender();
        System.out.println(banda.avanzar(5));
        System.out.println(anden);
    }
}
```

```
DEBE SALIR
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 C
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 C
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 C ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
```

En el ciclo 2, **un solo** sensor en alerta detiene todo. En el ciclo 3 la estación funciona sin la báscula, y la báscula regresa con `78%`: es **el mismo objeto**, con la batería que ya había gastado.

Si algo truena

No compila

- **`SensorPresencia is not abstract and does not override abstract method hayAlerta() in Sensor`**: a tu sensor le falta uno de los métodos abstractos del padre. Una hija concreta tiene que escribir **todos**: `medir()` y `hayAlerta()`.
- **`Sensor is abstract; cannot be instantiated`**: `Sensor` es abstracta: no hay sensores a secas, solo de presencia, de peso o de temperatura.
- **`call to this must be first statement in constructor`**: el `this(...)`, igual que el `super(...)`, tiene que ser la primera línea del constructor.
- **`senal has private access in Sensor`**: la señal es privada de `Sensor`. Las hijas la leen con `getSenal()`, que es `protected`.
- **`Semaforo is not public in actuadores; cannot be accessed from outside package`**: a la clase le falta `public`. Sin modificador solo la ve su paquete, y `Main` vive en `estacion`.
- **`Bateria is not public in dispositivos; cannot be accessed from outside package`**: quisiste crear una batería desde otro paquete. La batería la crea `Dispositivo` y nadie más.
- **`leer() in SensorPeso cannot override leer() in Sensor`**: `leer()` es `final` en `Sensor`: las hijas no la sobrescriben, solo escriben `medir()` y `hayAlerta()`.
- **`cannot find symbol`**: falta un `import`. Cada clase de otro paquete se importa por su nombre completo, por ejemplo `import actuadores.Alarma;`.

Compila, pero la salida no es la de la guía

- **En el ciclo 2 la banda sigue avanzando**: tu `ciclo()` se queda con la alerta del último sensor. Lee **todos** los sensores primero y, si **alguno** dio alerta, hay alerta.
- **En el ciclo 3 vuelve a salir `S2` y falta `S3`**: tu `retirar` resta el contador pero no recorre los demás a la izquierda.
- **Sale `Regresa: null`**: tu `retirar` borra el sensor antes de guardarlo para devolverlo.
- **La alarma dice `volumen 15`**: el constructor de `Alarma` no valida.

Paso 4 · La entrega

QUÉ	DETALLE
Cuándo	**Viernes 23 de octubre, 23:59.** Sin IA, como siempre.
Con quién	En pareja (o de tres si alguien se queda solo). **Cada integrante sube todo en su propia carpeta.**
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p08/`. Solo push.
Qué	Tu carpeta `src`, la carpeta `out` (los `.class`) y una **bitácora en PDF**

La bitácora lleva: los integrantes (nombre y número de cuenta), la foto o captura de **tu diagrama**, la captura de la **salida de `Main`**, y estas tres respuestas con tus palabras:

1. **Sobrecarga contra sobrescritura:** un ejemplo de cada una sacado de tu código, y la diferencia.
2. **Composición contra agregación:** ¿por qué la `Bateria` es composición y el sensor en la `Estacion` es agregación? Piensa en quién hace el `new` y en qué le pasó a `S2` en el ciclo 3.
3. **Modificadores:** agrega en `Main` la línea `bascula.gastar(5);`, copia el error y explica por qué `SensorPeso` sí puede usar `gastar` y `Main` no. Bórrala antes de entregar.

Cómo se califica, sobre 10:

QUÉ REVISO	PUNTOS
Diagrama: las 12 cajas con sus símbolos, herencia, ◆, ◇ y multiplicidad	2
Corre y la salida es la de la guía	2
Constructores: `super(...)` y `this(...)` donde dicen las fichas	2
Sobrecarga y sobrescritura: `instalar`, `avanzar` y los `@Override`	1.5
Modificadores y paquetes: como en las fichas y el mapa	1.5
Bitácora: las tres respuestas	1

```
git checkout entregas_apellido_nombre
git pull origin main
mkdir -p entregas/apellido_nombre/p08
# copia aquí src, out y la bitácora en PDF
git add entregas/apellido_nombre/p08
git commit -m "P8: herencia, segunda parte"
git push
```

Si los `.class` no aparecen en `git status`, usa `git add -f`. El código tiene que ser suyo; si algo no sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM

Prof. Oscar Manuel Ruiz Hurtado · Práctica 8 · Herencia, segunda parte (MADO-21, guía 8: constructores sobrecargados, `this` y `super`, sobrecarga y sobrescritura) · octubre de 2026